ProfsAgent Phase 2 · Deliverable A: simulations, tools and how they plug in

Owner: Amit · **Status:** v0.1, 27 Oct 2026. Five simulations are built and tested; the work is paused while the Phase 1 platform is built. · **Read with:** `docs/04_phase2_UI_integration_guide.md` (Vihan) and `docs/03_phase2_C_learning_resources_and_experts_plan.md`.

1. Goal and the four tests

For a lecture topic, give students an **interactive simulation** that opens in a browser, together with **the code behind it**. It must be **correct**, **checked automatically**, and **approved by the professor** before students see it. The focus is undergraduate courses at IIIT-D.

A tool or library is used only if it passes all four tests:

1. **Runs in the browser.** Nothing to install for students.
2. **An AI can write it reliably.** That means code or text formats with many public examples, not point-and-click software.
3. **It can be checked automatically.** It must be deterministic and run headless.
4. **Its licence is free for teaching,** and for other colleges later.

The core rule: the answers come from real code, never from an AI writing them down. The code computes every step and the page only draws those steps. A second, independent method re-checks the answer, and the page shows the result as "✓ Answer checked".

2. What is built and tested so far

Simulation	What students can do	Independent check of the answer
Kosaraju (strongly connected components)	Step through both depth-first passes and the reversed graph; edit the graph	Mutual reachability of every pair of nodes
Dijkstra shortest paths	Presets that include a negative-edge trap and an unreachable node; predict-the-next-node questions	Bellman–Ford
Round Robin CPU scheduling	Set the processes and the time quantum; see the Gantt chart and waiting/turnaround times	A separate unit-by-unit scheduler (808 cases)
Valid parentheses (stack)	Type any bracket string; watch each push and pop; get Valid / Not valid with the exact reason	Repeated pair removal (3,012 strings)
B+ tree with cascading splits and merges	Type the page size and keys; choose the split rules (order or max keys, left or right bias, copy-up)	Full rule checker after every operation (630 random runs, 0 violations); matches a textbook figure at all 5 stages

What all five share:

- one HTML file that works offline;
- Back, Next, Play and a step slider;
- the code with the current line highlighted;
- prediction questions;
- fuzz tests against a Python reference;
- a headless click-through at laptop (1280 px) and phone (390 px) width;
- MP4 recordings for slides.

OpenMAIC comparison. OpenMAIC (open source, MIT) builds a whole lesson from a topic, including slides, a voice teacher and simulations. We tested it on the same three topics, and **none of its 8 simulations was fully correct**:

- **Dijkstra** stopped after the first node.
- **Round Robin** reported "complete" at 4.13 ms with an empty Gantt chart.
- **Kosaraju** would find 6 strongly connected components instead of 2, and its puzzle marked correct answers wrong.
- **Two scenes crashed** because a library failed to load from the internet.

Why: a single AI call writes the example, the algorithm and the answers, and the only check is for syntax errors.

Decision: use OpenMAIC only for the lesson layer (slides, narration, quizzes, export). Never use it as the source of a simulation's answers.

3. How a simulation is produced

1. **Reuse first.** Look in our verified simulations and in the catalogue of trusted online simulations (section 5). The topic is matched through the Layer A topic embeddings.
2. **Template plus truth engine.** If nothing fits, pick a locked page template and connect a truth engine (real code) that produces every step. The AI fills in only the topic-specific part.
3. **New generation.** Only when no template fits. Claude writes the new code, and it must pass the same checks. Failures are sent back for a fix, a few times at most.
4. **Checks.**
    - The answer is verified by a second method.

- A headless browser click-through, at laptop and phone size, confirms: no errors, the picture changes when Play is pressed, sliders change the result, and Reset resets.
- Nothing loads from the internet.

5. **Professor approval.** The professor uses approve / replace / reject, giving a reason for the last two (same rule as Vihan's guide, S5).
6. **Stored for reuse.** The approved simulation becomes a node in Neo4j linked to its `Topic`, so every course that teaches the same topic gets it for free.

4. Final toolkit by layer

Layer	Tools	Cost
Truth engine (computes every step)	Plain JavaScript in the page; SimPy, SciPy, NumPy, NetworkX for Python-native topics; Pyodide only when Python must run live in the browser	Free
Drawing	Our SVG template (graphs, trees, stacks, queues, Gantt charts) · p5.js (motion, waves, particles) · JSXGraph (maths) · Matter.js (2D physics; answers checked against formulas with a tolerance) · D3.js / Plotly (data) · Three.js (3D only, fixed camera) · KaTeX (formulas)	Free
Template shell	Back, Next, Play, step slider, explanation panel, code with the current line highlighted, prediction questions, phone layout	Free
Verification	Playwright click-through at two widths; a second method for every answer; random fuzz cases (e.g. Hypothesis); a vision-model critic only as a last, advisory step	Free
Videos and slides	Our Playwright + ffmpeg recorder · Manim (maths videos) · Mermaid (diagrams) · Reveal.js (web slides with live simulations) · python-pptx (PowerPoint export with the MP4 and a link)	Free
AI	Claude Opus 5.5 to build new templates ($4 / $20 per million input/output tokens, about $1–2 per template, built once). A cheaper model such as Claude Haiku 5.5 ($0.10 / $0.50) is an option for routine filling-in; not decided yet	Small, paid
Storage	Approved simulations stored against their `Topic` in Neo4j	Free

Rules the pipeline keeps:

- Pinned library versions are **bundled with the file**; no CDN links (CDN loading is what crashed OpenMAIC's scenes).
- Plain CSS, no build step.
- Wherever textbooks differ, the setting is shown on the page. The B+ tree's order and split rules are an example; they let the output match the professor's notes.

Not used:

- **ANSYS, COMSOL, AnyLogic, Fusion 360, Tinkercad, SketchUp, FreeCAD:** desktop tools, not usable in a browser, and impossible to generate or check automatically.
- **Blender:** too heavy.
- **NVIDIA Kaolin:** a research library, not a teaching tool.
- **Mafs:** needs React and a build step.
- **Optional only:**
  - MATLAB/Simulink, only if a campus licence exists;
  - Cytoscape.js, for large user-drawn graphs;
  - Docker, on a server.

5. Reuse before building: trusted online simulations

Source	Covers	Use and licence note
PhET	Physics, chemistry, maths	Embed or link. Versions before March 2026 are CC BY 4.0; check newer versions before any commercial use
VisuAlgo	Data structures and algorithms	**Link only.** Its terms forbid copying it or making variants; embedding needs the author's permission
Falstad circuits · **CircuitVerse**	ECE circuits, digital logic	Falstad is GPL and supports iframe embedding; CircuitVerse has a per-project embed link
Python Tutor	Code running step by step	Embedding supported; licence still to be checked
TensorFlow Playground, **CNN Explainer**, **Seeing Theory**	ML, probability	Link

Which tool for which course area:

Area	Tools
CSE / CSAI	SVG templates with JS or Pyodide, SimPy for queues; reuse VisuAlgo and Python Tutor
ML	Plotly with NumPy; TF Playground

Area	Tools
ECE	Falstad, CircuitVerse, python-control, p5.js for signals
Maths	JSXGraph, Plotly, Manim
Bio / social science	Mesa or NetLogo Web
Design / graphics	Three.js, p5.js
Intro physics	PhET, Matter.js

6. How simulations plug into the Phase 2 delivery flow (Vihan's guide)

Vihan's guide gives A (Amit) the screens **and** the simulations. Simulations fit the same pattern as C's resources: the professor makes a choice first, a job runs, cards come back, and the professor reviews them. **Section 6.1 is a proposal; nothing in Vihan's `delivery/` code changes without his OK.**

Guide screen	What is added for simulations
S2 · delivery choices	A sixth multi-select option: **"Add an interactive simulation"** (proposed survey key `simulation: yes/no`, plus an optional `sim_note`, e.g. "use the textbook's order-3 B+ tree"). It is shown only when the lecture's topic suits a simulation; otherwise it is hidden. Choices-first still applies: nothing is built before the choice is saved
S3 · progress	One more row: "Finding or building a simulation…" (a background job; reuse takes seconds, building takes minutes)
S4 · results	A **Simulation (A)** tab. Each card shows: title, a source badge (*Ours, verified* · *PhET* · *VisuAlgo* · *Newly built*), licence, the learning outcomes it covers, the checks passed (✓ answer checked · ✓ laptop and phone · ✓ works offline), a live preview, and any warnings in yellow
S5 · review	The same Approve / Replace / Reject, with a reason required for the last two. Proposed `review.yaml` kind: `simulation`. Replace takes the next candidate (e.g. the PhET entry if ours is rejected)
S7 · student view	Approved simulations only, with their code view and a "try your own input" box. No scores or warnings
S8 · feedback and health	Students and professors rate simulations 1–5 (`feedback.yaml`). The weekly link-health job also re-checks embedded external simulations

6.1 Proposed data contract (for agreement with Vihan and Parth)

- **Survey keys (in `survey.yaml`):** `simulation`, `sim_note`.
- **Routing:** a new row in `survey.tasks()` with owner A.
- **Files A writes:**
  - `runs/<id>/delivery/simulations/<lecture_id>.json` with `items[] = {title, source, url or file, licence, covers_los[], checks{answer, browser, offline}, warnings[], rank}`;
  - the simulation files themselves, in `runs/<id>/delivery/simulations/<lecture_id>/`.
- **Neo4j:**
  - `(:Delivery:Simulation)-[:FOR_LECTURE]->(:D_Lecture)`
  - `(:Simulation)-[:TEACHES]->(:Topic)` for reuse across courses.

6.2 Links with B and C

- **C (Vihan).** C's "practice / interactive" search already finds visualisers such as VisuAlgo and PhET (plan 03, §3.1). A uses C's results as reuse candidates, and the UI removes duplicates, so the professor never reviews the same simulation twice.
- **B (Parth).** B's generated slides embed approved simulations: live in Reveal.js web slides, and as the recorded MP4 with a link in PowerPoint. Simulations stay A's responsibility; B only places them.

7. Next steps

1. **Now:** build the Phase 1 platform (local app with the real pipeline and the three gates). The simulation work stays on hold until then.
2. **Then:**
   - write the "simulation skill" (rules, templates and check scripts) and the trusted-simulation catalogue;
   - add 2–3 more templates (sorting/arrays, queues with SimPy, p5.js motion);
   - add the S2 option and the S4 tab to the UI once Vihan agrees to 6.1.
3. **Open questions:**
   - (a) Vihan: the survey key and review kind;
   - (b) Parth: how B embeds simulations in slides;
   - (c) whether to use a cheaper model for routine filling-in.